

The Complete Guide to Building with Astro from Scratch

This guide helps you establish a production-grade Astro project. You learn how to design a scalable directory architecture, build type-safe content pipelines across multiple collections, configure unified dynamic routing, and load heavy client-side libraries conditionally with zero performance overhead.

Phase 1: Project Setup & Directory Spaces

Astro is a modern web compiler designed for building content-focused websites with a focus on speed. Its core architecture compiles your templates to completely static, zero-client-side JavaScript HTML by default.

Architectural Blueprint: Why Astro? (Astro vs. Eleventy)

When choosing a static site architecture, understanding the engineering philosophy of your compiler is critical. While both Astro and Eleventy (11ty) excel at compiling static assets, they manage data pipelines and compilation scopes differently:

- **Data Scope and Type Integrity:** Eleventy uses an open-ended, schema-less Data Cascade that merges directory-level JSON files, layout frontmatter, and file-specific YAML properties into a single template namespace. Astro isolates variables strictly inside a typed, schema-validated namespace (`project.data`).
- **Type Safety:** Astro generates internal TypeScript declarations on every compilation run, providing complete IDE autocomplete. Eleventy is natively untyped and requires custom helper scripts or manual validations to achieve similar integrity.
- **Modern Integration Hooks:** Astro is a fully featured Vite compiler. It allows you to bundle modern frameworks (React, Svelte) as scoped islands of interactivity and use modern CSS processors (Tailwind v4) out-of-the-box.

The Separated Concerns Pattern

In modern Astro, where your content lives is completely divorced from how the framework routes it. Instead of placing duplicate `index.astro` files inside every content folder (a legacy co-located pattern), you organize raw markdown files into subfolders inside `src/content/` and use a single wildcard route in `src/pages/`.

```
src/
├── components/
│   └── Welcome.astro      # Welcome component
├── content/
│   ├── ai-projects/      # Raw Content Space (Data Layer)
│   └── code-projects/
```

```

|   └─ id-projects/
├── layouts/
|   └─ Layout.astro      # Global layout wrapper
└── pages/
    ├── index.astro      # Root homepage
    ├── [collection]/    # Single Source of Truth (Routing Layer)
    │   ├── index.astro  # Handles all collection index pages
    │   └─ [...id].astro # Handles all individual project pages

```

This structure ensures strict adherence to the DRY (Don't Repeat Yourself) principle. You write your rendering logic exactly once. If you update the layout, the changes instantly ripple across all project categories.

Initialize the Astro Project

To start a new Astro project, run the installation wizard in your terminal. To create a strict TypeScript project using the empty template without interactive prompts, run:

```
npm create astro@latest my-astro-site -- --template empty --typescript strict --install --git
```

Phase 2: Defining the Content Schema (Zod)

Content collections allow you to organize markdown or MDX files into structured data models. Astro Content Collections use Zod, a TypeScript-first schema declaration and validation library.

Schema Validation and Default Values

Zod inspects your content frontmatter at build time. If an entry misses a required field or contains mismatched types, Astro stops the build and reports the exact validation error.

To prevent builds from crashing on legacy files that lack certain fields, use forgiving schema modifiers like `.optional().default()` or `.catch()`.

Create your collection configuration file (`src/content.config.ts`) and define schemas for your three project directories:

```
// src/content.config.ts
import { defineCollection } from 'astro:content';
import { z } from 'astro/zod';
import { glob } from 'astro/loaders';

// Define a forgiving, flat schema that safely handles missing data
const projectSchema = z.object({
  title: z.string().catch('Untitled Project'),

```

```

author: z.string().optional().default('Carlos'),
date: z.coerce.date().catch(() => new Date()), // Converts string to native JS Date
language: z.string().optional(),
desc: z.string().optional(),
draft: z.boolean().optional().default(false),
baseline: z.boolean().optional().default(false),
colorjs: z.boolean().optional().default(false),
youtube: z.boolean().optional().default(false),
vimeo: z.boolean().optional().default(false),
mavo: z.boolean().optional().default(false),
mermaid: z.boolean().optional().default(false),
});

const aiProjects = defineCollection({
  loader: glob({ pattern: '**/[^_]*.{md,mdx}', base: './src/content/ai-projects' }),
  schema: projectSchema,
});

const codeProjects = defineCollection({
  loader: glob({ pattern: '**/[^_]*.{md,mdx}', base: './src/content/code-projects' }),
  schema: projectSchema,
});

const idProjects = defineCollection({
  loader: glob({ pattern: '**/[^_]*.{md,mdx}', base: './src/content/id-projects' }),
  schema: projectSchema,
});

// Export all three collections explicitly
export const collections = {
  'ai-projects': aiProjects,
  'code-projects': codeProjects,
  'id-projects': idProjects,
};

```

Phase 3: The Data Pipeline & Dynamic Routing

A dynamic route is defined by wrapping a directory or filename inside square brackets, such as `[collection]/[...id].astro`. To build static sites, dynamic pages must export an asynchronous `getStaticPaths()` function that queries the data store and maps out the generated URLs.

1. The Unified Collection Index Template

This template catches routes like `/ai-projects/` and `/code-projects/`. It uses a unified TypeScript type alias (`ProjectEntry`) to prevent union collapse when sorting diverse collections, and it implements `Promise.allSettled` to keep the production build alive even if one collection fails to load.

Create this file at `src/pages/[collection]/index.astro`:

```
---
// src/pages/[collection]/index.astro
import { getCollection } from 'astro:content';
import type { CollectionEntry } from 'astro:content';

// 1. Define a unified type alias to prevent TypeScript union collapse
type ProjectEntry =
  | CollectionEntry<'ai-projects'>
  | CollectionEntry<'code-projects'>
  | CollectionEntry<'id-projects'>;

export async function getStaticPaths() {
  const isDev = import.meta.env.DEV;
  // Use 'collection' nomenclature to match the filename parameters
  const uniqueCollections = ['ai-projects', 'code-projects', 'id-projects'] as const;

  let allProjects: ProjectEntry[] = [];

  if (isDev) {
    // Strict Mode: Catch frontmatter errors instantly during local dev
    const [ai, code, id] = await Promise.all([
      getCollection('ai-projects'),
      getCollection('code-projects'),
      getCollection('id-projects'),
    ]);
    allProjects = [...ai, ...code, ...id];
  } else {
    // Forgiving Mode: Keep the production build alive if one collection fails
    const results = await Promise.allSettled([
      getCollection('ai-projects'),
      getCollection('code-projects'),
      getCollection('id-projects'),
    ]);

    allProjects = results.flatMap((result) =>
```

```

    result.status === 'fulfilled' ? (result.value as ProjectEntry[]) : []
  );
}

// Safely filter out drafts using the flat schema
const publicProjects = allProjects.filter((project) => !project.data.draft);

return uniqueCollections.map((collection) => {
  const filteredProjects = publicProjects
    .filter((project) => project.collection === collection)
    .sort((a, b) => {
      // Safe Date to ISO string conversion for the Temporal API
      const dateA = a.data.date.toISOString().split('T')[0];
      const dateB = b.data.date.toISOString().split('T')[0];
      return Temporal.PlainDate.compare(
        Temporal.PlainDate.from(dateB),
        Temporal.PlainDate.from(dateA)
      );
    }) as ProjectEntry[];

  return {
    // The param key MUST be 'collection' to match the [collection] directory/file name
    params: { collection },
    props: { projects: filteredProjects },
  };
});
}

interface Props {
  projects: ProjectEntry[];
}

// Destructure 'collection' to match the params returned by getStaticPaths
const { collection } = Astro.params;
const { projects } = Astro.props;
---

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>{collection.replace('-', ' ')}</title>

```

```
<style>
:root {
  --color-bg: #0f172a;
  --color-card-bg: #1e293b;
  --color-text: #f8fafc;
  --color-accent: #38bdf8;
  --color-muted: #94a3b8;
}
body {
  margin: 0;
  font-family: system-ui, -apple-system, sans-serif;
  background-color: var(--color-bg);
  color: var(--color-text);
  line-height: 1.6;
  padding: 4rem 1rem;
}
main {
  max-width: 800px;
  margin: 0 auto;
}
.back-link {
  color: var(--color-accent);
  text-decoration: none;
  display: inline-block;
  margin-bottom: 1.5rem;
  font-weight: 500;
  transition: transform 0.2s ease;
}
.back-link:hover {
  transform: translateX(-4px);
}
.title-section {
  border-bottom: 2px solid var(--color-card-bg);
  padding-bottom: 1.5rem;
  margin-bottom: 2.5rem;
}
.title-section h1 {
  font-size: 2.75rem;
  margin: 0 0 0.5rem 0;
  text-transform: capitalize;
}
.title-section p {
  color: var(--color-muted);
}
```

```
margin: 0;
font-size: 1.125rem;
}
.posts-grid {
display: flex;
flex-direction: column;
gap: 1.75rem;
}
.post-card {
background-color: var(--color-card-bg);
border: 1px solid rgba(255, 255, 255, 0.05);
padding: 1.75rem;
border-radius: 12px;
transition: transform 0.25s cubic-bezier(0.16, 1, 0.3, 1), border-color 0.2s ease;
}
.post-card:hover {
transform: translateY(-2px);
border-color: var(--color-accent);
}
.post-card h2 {
margin: 0 0 0.75rem 0;
font-size: 1.5rem;
}
.post-card h2 a {
color: var(--color-text);
text-decoration: none;
}
.post-card h2 a:hover {
color: var(--color-accent);
}
.post-meta {
font-size: 0.875rem;
color: var(--color-muted);
margin-bottom: 1rem;
display: flex;
gap: 0.5rem;
align-items: center;
}
.post-description {
margin: 0;
color: var(--color-muted);
font-size: 1rem;
}
```

```

.no-posts {
  text-align: center;
  padding: 4rem 2rem;
  background-color: var(--color-card-bg);
  border-radius: 12px;
  color: var(--color-muted);
}
</style>
</head>
<body>
<main>
  <nav aria-label="Breadcrumb">
    <a href="/" class="back-link">← Back to Home</a>
  </nav>

  <header class="title-section">
    <h1>{collection.replace('-', ' ')}</h1>
    <p>A collection of projects covering {collection.replace('-', ' ')} concepts, practices, and
pipelines.</p>
  </header>

  {projects.length === 0 ? (
    <div class="no-posts">
      <p>No projects published in this collection yet.</p>
    </div>
  ) : (
    <section class="posts-grid" aria-label={` Projects in ${collection}`}>
      {projects.map((project) => {
        const projectDate = project.data.date.toISOString().split('T')[0];
        return (
          <article class="post-card">
            <h2>
              <a href={`/${project.collection}/${project.id}`}>{project.data.title}</a>
            </h2>
            <div class="post-meta">
              <span>By {project.data.author}</span>
              <span>•</span>
              <time datetime={projectDate}>
                {project.data.date.toLocaleDateString("en-US", { dateStyle: "long" })}
              </time>
            </div>
            {project.data.desc && (
              <p class="post-description">{project.data.desc}</p>

```



```

    })
  </article>
  );
  }}}
</section>
})
</main>
</body>
</html>

```

2. The Unified Project Details Template

This template catches deep routes like `/ai-projects/machine-learning-overview`. It receives the exact entry mapped from `getStaticPaths` and renders the markdown body using the `render()` utility.

Create this file at `src/pages/[collection]/[...id].astro`:

```

---
import { getCollection, render } from 'astro:content';
import type { CollectionEntry } from 'astro:content';

type ProjectEntry =
  | CollectionEntry<'ai-projects'>
  | CollectionEntry<'code-projects'>
  | CollectionEntry<'id-projects'>;

export async function getStaticPaths() {
  const isDev = import.meta.env.DEV;

  let allProjects: ProjectEntry[] = [];

  if (isDev) {
    const [ai, code, id] = await Promise.all([
      getCollection('ai-projects'),
      getCollection('code-projects'),
      getCollection('id-projects'),
    ]);
    allProjects = [...ai, ...code, ...id];
  } else {
    const results = await Promise.allSettled([
      getCollection('ai-projects'),
      getCollection('code-projects'),

```

```

    getCollection('id-projects'),
  ]);
  allProjects = results.flatMap((result) =>
    result.status === 'fulfilled' ? (result.value as ProjectEntry[]) : []
  );
}

const publicProjects = allProjects.filter((project) => !project.data.draft);

return publicProjects.map((project) => ({
  params: {
    collection: project.collection,
    id: project.id,
  },
  props: { project },
})));
}

interface Props {
  project: ProjectEntry;
}

```

```

const { project } = Astro.props;
const { Content } = await render(project);
const projectDate = project.data.date.toISOString().split('T')[0];
---
```

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>{project.data.title}</title>
    <style>
      :root {
        --color-bg: #0f172a;
        --color-card-bg: #1e293b;
        --color-text: #f8fafc;
        --color-accent: #38bdf8;
        --color-muted: #94a3b8;
      }

      body {

```

```
margin: 0;
font-family: system-ui, -apple-system, sans-serif;
background-color: var(--color-bg);
color: var(--color-text);
line-height: 1.6;
padding: 4rem 1rem;
}
```

```
main {
  max-width: 800px;
  margin: 0 auto;
}
```

```
.back-link {
  color: var(--color-accent);
  text-decoration: none;
  display: inline-block;
  margin-bottom: 2rem;
  font-weight: 500;
  transition: transform 0.2s ease;
}
```

```
.back-link:hover {
  transform: translateX(-4px);
}
```

```
article {
  background-color: var(--color-card-bg);
  border: 1px solid rgba(255, 255, 255, 0.05);
  padding: 2.5rem;
  border-radius: 16px;
}
```

```
h1 {
  font-size: 3rem;
  margin: 0 0 1rem 0;
  line-height: 1.2;
}
```

```
.post-meta {
  color: var(--color-muted);
  display: flex;
  gap: 0.75rem;
```

```
align-items: center;
margin-bottom: 2.5rem;
padding-bottom: 1.5rem;
border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}
```

```
.content {
  font-size: 1.125rem;
}
```

```
.content :global(h2) {
  color: var(--color-accent);
  margin-top: 2.5rem;
}
```

```
.content :global(a) {
  color: var(--color-accent);
  text-decoration: none;
}
```

```
.content :global(a:hover) {
  text-decoration: underline;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<main>
```

```
<a href={`/${project.collection}`} class="back-link">← Back to {project.collection.replace('-', '
')}}</a>
```

```
<article>
```

```
<h1>{project.data.title}</h1>
```

```
<div class="post-meta">
```

```
<span>By {project.data.author}</span>
```

```
<span>•</span>
```

```
<time datetime={projectDate}>
```

```
{project.data.date.toLocaleDateString("en-US", { dateStyle: "long" })}
```

```
</time>
```

```
</div>
```

```
<div class="content">
```

```
<Content />
```

```
</div>
```

```
</article>
</main>
</body>
</html>
```

Phase 4: Extending with Custom Content Loaders

While the `glob()` loader handles local Markdown files beautifully, you often need to ingest structured data from external REST APIs or headless CMS platforms. You can use Astro's Content Layer API to fetch, parse, and store external data natively by writing a custom loader.

To set up an external authors collection that pulls data from a remote endpoint at build time, follow these steps:

1. Define the Custom Loader

Add the authors collection definition to your `src/content.config.ts` file alongside your existing markdown projects. The custom loader fetches the remote data and saves it directly into Astro's internal data store.

```
// Define an authors collection using a custom API loader
const authors = defineCollection({
  loader: {
    name: 'authors-api-loader',
    load: async ({ store }) => {
      // Fetch data from your external REST API
      const response = await
fetch('https://jsonplaceholder.typicode.com/users')(https://jsonplaceholder.typicode.com/users)
    );
      const users = await response.json();

      for (const user of users) {
        // Save each item to the Astro store using a unique ID
        store.set({
          id: user.username.toLowerCase(),
          data: {
            name: user.name,
            email: user.email,
            website: user.website,
          },
        });
      }
    }
  }
});
```

```

},
// Validate the incoming API data to ensure type safety
schema: z.object({
  name: z.string(),
  email: z.string().email(),
  website: z.string(),
}),
});

```

2. Export the Collection

For Astro to recognize the new collection, you must export it. Append authors to your collections export object at the bottom of the `src/content.config.ts` file:

```

export const collections = {
  'ai-projects': aiProjects,
  'code-projects': codeProjects,
  'id-projects': idProjects,
  'authors': authors, // Make sure to export the external collection here
};

```

Phase 5: Dynamic Performance & Asset Optimization

Astro integrates surgical UI framework islands with dynamic client-side scripting to maintain a zero-JS baseline. By relying on peer dependencies, Astro prevents the duplication of core runtime engines like React or Svelte.

Consolidated Conditional Loading

You can use standard JSX-style conditional logic directly in your Layout files to dynamically compile, bundle, and execute local node packages only when specific frontmatter conditions evaluate to true.

By using Vite Code Splitting via `dynamic import()` statements inside standard client-side `<script>` tags, Vite compiles each package into a separate chunk. The browser executes this code only when the frontmatter flag evaluates to true.

```

---
// Conditionally run these scripts inside your layout based on frontmatter flags
const { mermaid, baseline } = project.data;
---

```

```

<!-- 1. Conditional Diagram Rendering (Mermaid.js) -->

```

```
{mermaid && (
  <script>
    const renderDiagrams = async (): Promise<void> => {
      const { default: mermaid } = await import('mermaid');
      mermaid.initialize({ startOnLoad: false, theme: 'dark' });
      await mermaid.run({
        nodes: document.querySelectorAll('.mermaid-diagram'),
      });
    };
    renderDiagrams();
  </script>
)}
```

<!-- 2. Conditional Analytics Rendering (Chart.js) -->

```
{baseline && (
  <script>
    const initCharts = async (): Promise<void> => {
      const { Chart, registerables } = await import('chart.js');
      Chart.register(...registerables);
      // Initialize charts...
    };
    initCharts();
  </script>
)}
```

Phase 6: Styling, Asset Delivery & SEO

Configuration & SEO (Resolving Sitemap Errors)

Integrations like @astrojs/sitemap require a definitive base URL to generate correct, absolute paths for search engines. Provide the site property inside your Astro configuration to resolve integration warnings and guarantee valid sitemaps.

Configure astro.config.mjs to include your URL and Tailwind v4 setup:

```
import { defineConfig } from 'astro/config';
import tailwindcss from '@tailwindcss/vite';
import sitemap from '@astrojs/sitemap';

export default defineConfig({
  // The 'site' property is required for sitemap and RSS URL resolution
  site: '[https://your-portfolio-domain.com](https://your-portfolio-domain.com)',
  integrations: [sitemap()],
```

```
vite: {  
  plugins: [tailwindcss()],  
},  
});
```

Tailwind CSS v4 (Vite-First Setup)

In Tailwind CSS v4, the architecture is fully optimized for Vite-based environments. You run Tailwind v4 directly inside Astro's Vite pipeline.

1. **Install Dependencies:**

```
npm install tailwindcss @tailwindcss/vite
```

2. **Import Tailwind Directive:** Create a global CSS file (`src/styles/global.css`) and reference Tailwind:

```
@import "tailwindcss";
```

Static Assets: `/public` vs. `src/assets/`

Choosing the correct asset location impacts build-time processing:

- **`src/assets/`:** Use this for images and CSS that you want Vite to minify, bundle, tree-shake, and cache-bust using content hashes.
- **`/public`:** Use this for stable paths (like `/fonts/font.woff2`) that require predictable, un-hashed URL routing, or for assets that bypass Vite compilation entirely.

Bypassing the Pipeline: Serving Raw Root HTML

If you need to serve raw HTML files from a root folder (like a `manuals/` directory for legacy documentation) and bypass Astro's processing entirely, copy them directly into the output build folder (`dist/`) during compilation. This provides flat, accessible public URLs without passing the files through any Astro layouts or bundlers.

1. Install the Static Copy Plugin

Use the `vite-plugin-static-copy` plugin to move root-level folders into the final build directory natively.

```
npm install -D vite-plugin-static-copy
```

2. Configure Vite Static Copy

Import the plugin and add it to Astro's underlying Vite configuration block in `astro.config.mjs`.

```
import { defineConfig } from 'astro/config';
```



```

import { viteStaticCopy } from 'vite-plugin-static-copy';
import tailwindcss from '@tailwindcss/vite';
import sitemap from '@astrojs/sitemap';

export default defineConfig({
  site: ['https://your-portfolio-domain.com'](https://your-portfolio-domain.com),
  integrations: [sitemap()],
  vite: {
    plugins: [
      tailwindcss(),
      viteStaticCopy({
        targets: [
          {
            // Path to your root folder relative to astro.config.mjs
            src: 'manuals/*',
            // Copies files directly into the root of your final build folder
            dest: './'
          }
        ]
      })
    ]
  },
});

```

3. Dynamically Scan and Link to Raw Assets

To generate a list of links automatically without hardcoding them, use the Node fs module directly inside an Astro component's frontmatter. Since these files bypass the Vite asset pipeline entirely, reading the directory at build time is the safest way to construct flat URLs that match your copied assets exactly.

```

---
// src/components/ManualsList.astro
import fs from 'node:fs/promises';
import path from 'node:path';

const manualsDir = './manuals';
let manualLinks: Array<{ url: string; title: string }> = [];

try {
  // Scan the root directory at build time
  const files = await fs.readdir(manualsDir);

```

```

manualLinks = files
  .filter((file) => file.endsWith('.html') || file.endsWith('.pdf'))
  .map((file) => {
    const title = path.basename(file, path.extname(file)).replace(/-/g, ' ');
    return {
      // Maps exactly to the static copy destination structure
      url: `/${file}`,
      title: title.charAt(0).toUpperCase() + title.slice(1)
    };
  });
} catch (error) {
  console.warn('Manuals directory not found or empty.');
```

```

<section>
  <h2>Legacy Reference Manuals</h2>
  {manualLinks.length === 0 ? (
    <p>No manuals available.</p>
  ) : (
    <ul>
      {manualLinks.map((link) => (
        <li>
          <a href={link.url}>{link.title}</a>
        </li>
      ))}
    </ul>
  )}
</section>
```

Phase 7: Production Readiness, Linting & Homepages

Internal Linking Rules

To ensure your links work across all pages of your site, always use root-relative paths (paths that start with a /).

- **Linking to a Collection:** `<a href=/{${project.collection}}>Back</a>` resolves safely to `/ai-projects/`.
- **Linking to a Project:** `<a href=/{project.collection/
{project.id}}>Read` resolves safely to `/ai-projects/my-post/`.

Omitting the leading / (e.g., `ai-projects/my-post`) causes the browser to evaluate the URL

relative to the user's current nested directory, resulting in broken 404 errors.

ESLint Configuration

Enforce code quality with ESLint Flat configs. Create an `eslint.config.mjs` file in the root of your project:

```
import eslint from '@eslint/js';
import tseslint from 'typescript-eslint';
import eslintPluginAstro from 'eslint-plugin-astro';

export default tseslint.config(
  eslint.configs.recommended,
  ...tseslint.configs.recommended,
  ...eslintPluginAstro.configs.recommended,
  ...eslintPluginAstro.configs['jsx-a11y-recommended'],
  {
    rules: {
      '@typescript-eslint/no-unused-vars': ['warn', { argsIgnorePattern: '^_' }],
    },
  }
);
```

The Root Homepage Template

The root `src/pages/index.astro` template acts as your landing page. You use this file to incorporate your site's main layouts and welcome components.

```
---
import Welcome from '../components/Welcome.astro';
import Layout from '../layouts/Layout.astro';
---

<Layout>
  <Welcome />
</Layout>
```